

MacroSnaps - Living Brief

Last updated: March 12, 2026 (Tooling session - update_commodity_stories.py built and verified; checks drift between current commodity prices and storyWrittenAtPrice in data.json; rewrites stories at all three levels for any commodity that has moved past threshold (10% for Natural Gas, 5% for all others); applies directly to data.json with no review step; bootstrapped all 9 commodities on first run March 12, 2026)

HOW TO USE THIS FILE

This file has two parts.

Part 1 is the master session prompt. Copy everything between the START and END markers and paste it as your first message in any new chat.

Part 2 is the detailed project reference. It lives below the prompt and is for Claude to read from the uploaded file. You do not need to paste it.

PART 1 - MASTER SESSION PROMPT

Copy from here...

You are helping me build and maintain MacroSnaps, a daily global macro and markets dashboard. Read this entire prompt before doing anything.

I am also uploading LIVING_BRIEF.md. Read it in full before responding. It contains the full project reference including architecture, current content state, key file locations, and working preferences.

Before we start, tell me:

- What you understand the current state of the project to be (3-4 sentences)
- Which session type this is, based on what I describe below
- Confirm you are ready

Session types and what to upload

Do NOT upload all files every session. The large files will fill the context window immediately.

- Tooling session (Git, validator, build script, infrastructure): upload LIVING_BRIEF.md + build.py
- UI session (tooltip, CSS, layout, JS, shell features): upload LIVING_BRIEF.md + macrosnaps-shell.html
- Content session (writing stories, updating metrics, editing data): upload LIVING_BRIEF.md + data.json
- Full build session (changes across files, final assembly): upload all four files, keep it short

If you are not sure which type applies, describe what you want to do and ask which files are needed before uploading anything.

Never upload macrosnaps-globe.html. It is a build output, not a source file.

Working preferences

CRITICAL - PLAN BEFORE BUILDING: Never write code or make any edit without first presenting a clear plan of exactly what will change and why. Wait for explicit approval before proceeding. The word "go" is the signal to proceed. This applies to every change, no matter how small it seems.

Think before building. On any non-trivial change, share your approach and flag concerns before writing code. Wait for me to say "go."

Make surgical edits. Change the minimum needed. Do not rewrite surrounding code unless it is broken.

After each edit, briefly explain what changed and why.

If something feels architecturally wrong, say so before doing it.

Never present a task as done until the build has run successfully and the output has been copied to the outputs folder.

Stories are written by AI

All per-metric stories (beginner, moderate, expert) are written by Claude, not by hand. When metric values change meaningfully, `update_stories.py` calls the Claude API and rewrites the relevant stories automatically. Do not write or edit stories manually unless specifically asked to.

Session rhythm

We work in focused chunks. At the end of each natural unit of work (a feature shipped, a content block done, a bug fixed), before starting the next thing, you update `LIVING_BRIEF.md` and make it available for download. Do not wait until the context window is full. Write it while everything is fresh.

Writing style (apply to every response)

Write in plain, natural English. Do not use em dashes or en dashes. Only use a standard hyphen (-) if a dash is genuinely needed. Prefer commas, periods, or parentheses instead. Before outputting any response, scan it for those characters. If found, rewrite those sentences. Output only the final corrected version. This rule applies to all responses including code comments and story content written into `data.json`.

Story writing style (apply to all metric stories in data.json)

The goal is for every story to read as if a knowledgeable human wrote it quickly for a smart friend. AI-sounding writing undermines the product. Apply all of the following rules to every story at every level.

- No em dashes or en dashes, ever. Use commas, periods, or parentheses instead.
- No passive voice where an active version is natural. "The Fed raised rates" not "rates were raised by the Fed."
- No hedging openers. Never start a sentence with "It is worth noting," "It is important to understand," or "This reflects the fact that."
- No AI-typical sentence starters. Do not begin consecutive sentences with "This metric," "This reflects," or "This suggests."
- Vary sentence length deliberately. Mix short punchy sentences with longer ones. Uniform sentence length is a tell.
- Write numbers as if they are real and specific. "Inflation hit 8.4%" reads better than "the inflation rate stands at 8.4%."
- No filler conclusions. Never end a story with "overall," "in summary," or "taken together."
- No committee language. Write as if explaining to a smart friend, not presenting a report.
- Before outputting any story, scan it against every rule above and rewrite any sentence that fails. Output only the final corrected version.

This style guide must also be included verbatim in the system prompt used by `update_stories.py` when calling the Claude API.

What I am working on today:

[describe your task here]

[...to here](#)

PART 1B - STORIES SESSION PROMPT (use this instead of the master prompt when doing a story rewrite session)

Copy from here...

You are helping me rewrite stale metric stories for MacroSnaps, a daily global macro and markets dashboard.

I am uploading two files: `LIVING_BRIEF.md` and `data.json`. Read both in full before doing anything. The brief contains the full project reference and story writing rules. The data file contains all current metric values and existing story text.

Your job this session:

Rewrite the stories listed in the "Known value/story drift" section of the brief. These are metrics where the value has been updated but the story text still references old numbers.

For each metric, rewrite all three levels (beginner, moderate, expert) so they are consistent with the current value field in data.json.

Before writing anything, present a plan:

- List exactly which country/metric pairs you will rewrite
- Confirm the current value you will write to for each one
- Wait for me to say "go"

Story writing rules (apply to every story at every level):

- No em dashes or en dashes. Use commas, periods, or parentheses instead.
- No passive voice where an active version is natural.
- No hedging openers ("It is worth noting", "It is important to understand", "This reflects the fact that").
- No AI-typical sentence starters. Do not begin consecutive sentences with "This metric," "This reflects," or "This suggests."
- Vary sentence length deliberately. Mix short punchy sentences with longer ones.
- Write numbers as real and specific. "Inflation hit 8.4%" not "the inflation rate stands at 8.4%."
- No filler conclusions ("overall," "in summary," "taken together").
- No committee language. Write as if explaining to a smart friend.
- Scan every story against these rules before outputting. Output only the final corrected version.

Story length guidelines:

- Beginner: 2-3 short sentences. No jargon. One plain-English explanation of what the number means for everyday life.
- Moderate: 3-4 sentences. Include one piece of context (historical comparison, regional comparison, or causal driver).
- Expert: 4-5 sentences. Include specific numbers, a directional signal, and one forward-looking implication.

Output format:

For each rewrite, output a JSON block that can be copied directly into data.json. Use this format exactly:

```
"story": {  
  "beginner": "...",  
  "moderate": "...",  
  "expert": "..."  
}
```

After all rewrites are done, output a summary table of every country/metric pair that was updated.

Then update LIVING_BRIEF.md to clear the rewritten entries from the "Known value/story drift" section and make it available for download.

Do not rewrite stories for the 4 data-void metrics (Equity Vol, Corp Spread, Sov CDS, FX Vol). These are moving to "not available" display state.

What I am working on today: Rewriting stale metric stories. All current values are in data.json.

[...to here](#)

PART 1C - COMMODITIES SESSION PROMPT (use this when updating commodity stories)

Copy from here...

You are helping me update commodity stories for MacroSnaps, a daily global macro and markets dashboard.

I am uploading two files: LIVING_BRIEF.md and data.json. Read both in full before doing anything.

Your job this session:

Review the current value field for each of the 9 commodities in data.json and compare it to what the existing stories say. Rewrite any stories where the current price has moved meaningfully since the story was last written (roughly 5% or more for most commodities, 10% for more volatile ones like Natural Gas).

The 9 commodities: Oil (WTI), Natural Gas, Gold, Silver, Copper, Wheat, Corn, Iron Ore, Lithium.

Before writing anything, present a plan:

- List which commodities have meaningful drift between value and story text
- Show the current value vs what the story references
- Wait for me to say "go"

Story writing rules: Same as metric stories. No em dashes or en dashes. No passive voice. No hedging openers. Vary sentence length. Write numbers as real and specific. No filler conclusions. No committee language.

Story length guidelines:

- Beginner: 2-3 sentences. What this commodity is and why its price matters to ordinary people.
- Moderate: 3-4 sentences. Include one driver of the current price level and one downstream effect.
- Expert: 4-5 sentences. Include specific price, directional trend, supply/demand driver, and one forward implication.

Output format: Same JSON block format as metric stories. After all rewrites, output a summary of what changed.

Then update `LIVING_BRIEF.md` to note the commodity story update date and make it available for download.

What I am working on today: Reviewing and updating commodity stories based on current prices in `data.json`.

[...to here](#)

PART 2 - PROJECT REFERENCE

What the project is

MacroSnaps is a daily dashboard that makes global macro and market data accessible to everyone, from curious beginners to seasoned professionals.

Each day, we generate concise snapshots for 12 major economies: the G7 + BRICS. Every card includes key macro forecasts, live market data, AI-generated story bullets, and a weather icon that tells you the economic outlook at a glance.

The product is pre-launch. The architecture is intentionally simple. The goal right now is to get the product right before making it scalable.

Live URL: <https://ralphlazar.github.io/macrosnaps/macrosnaps-globe.html>

GitHub repo: <https://github.com/ralphlazar/macrosnaps> (branch: master)

The 4 source files

File	What it is	Touch it when
<code>data.json</code>	All content and data	Updating metrics, writing stories, changing values
<code>macrosnaps-shell.html</code>	The entire app shell (no data)	Changing UI, layout, CSS, JS logic
<code>build.py</code>	The assembly line script	Changing build logic or validation rules
<code>macrosnaps-globe.html</code>	The built output	Never - this is generated, not edited

All files live in `~/Downloads/macrosnaps/`.

To build: run `python3 build.py` from inside `~/Downloads/macrosnaps`. The output file is written to the same folder automatically.

To validate only (no files written): run `python3 build.py --validate-only`. Use this before editing `data.json`.

Never modify `_frozen_historical` or `_frozen_weatherGrid` inside any country in `data.json` by hand. To restore or update them, run `refetch_historical.py` (see below). Exception: the 2026F forecast appending and Current Account %

GDP conversion done on March 10 were deliberate one-time corrections and are now baked in.

Daily bash ritual (run every morning)

This is the complete workflow for a standard data update day. Copy and run these commands in order.

Step 1. Go to the project folder

```
cd ~/Downloads/macrosnaps
```

Step 2. Preview the sheet sync

This fetches your Google Sheet and shows what will change in `data.json`. Nothing is written yet.

```
python3 sync_sheet.py
```

Read the output. If the changes look correct, continue. If something looks wrong, fix the sheet first and re-run this step.

Step 3. Apply the sheet sync

This writes the macro metric changes to `data.json`.

```
python3 sync_sheet.py --apply
```

Step 4. Fetch live market data

This pulls Stock Market YTD, 10Y Bond Yield, Yield Curve, FX pairs, Equity Vol, Corp Spread, Sov CDS, FX Vol, and all 9 commodity prices from Yahoo Finance and FRED and writes them into `data.json`.

```
python3 fetch_market_data.py
```

Always run a dry run first on any new machine or after a long gap:

```
python3 fetch_market_data.py --dry-run
```

Step 5. Update commodity stories where prices moved

This checks each commodity's current price against the price recorded when the story was last written. Any commodity that has moved past its threshold (10% for Natural Gas, 5% for all others) gets a fresh story written at all three levels. If nothing has moved, it exits in under a second.

```
python3 update_commodity_stories.py
```

Step 6. Rewrite stories where values moved

This diffs `data.json` against the last git commit, identifies metrics that changed past threshold, calls the Claude API, and rewrites stories for those metrics at all three levels.

```
python3 update_stories.py
```

Step 7. Draft and review country + global headlines

This calls the Claude API with web search enabled and drafts fresh country-level story bullets and global stories. It writes a preview file but does NOT touch `data.json`.

```
python3 update_headlines.py
```

Wait for the draft to complete (8-10 minutes, 13 API calls). Then open `headline_review.html` in your browser, load the draft file, review and edit each country and global, approve, and export `stories_approved_YYYY-MM-DD.json`.

Then apply the approved draft:

```
python3 update_headlines.py --apply stories_approved_YYYY-MM-DD.json
```

Step 8. Build the output file

This validates `data.json`, assembles `macrosnaps-globe.html`, and saves a dated backup.

```
python3 build.py
```

The build must say `BUILD SUCCESSFUL` before you continue. If it fails, do not push.

Step 9. Commit and push

```
git add -A && git commit -m "Daily update $(date +%Y-%m-%d)"
git push origin master
```

Step 10. Verify the live site

Wait about 60 seconds, then open:

<https://ralphlazar.github.io/macrosnaps/macrosnaps-globe.html>

What the Google Sheet controls

The sheet is the single source of truth for the 6 macro metrics per country. These values are forecast-based and change infrequently. Update them in the sheet when consensus forecasts change, then run the daily ritual.

Important: The value field for macro metrics holds the year-end forecast from the Google Sheet, not the current live reading. For example, USA Policy Rate shows 3.25% (year-end forecast) even though the Fed funds rate is currently higher. Stories should reference both the current reading and the forecast where relevant.

Important: When a forecast changes in the sheet, `sync_sheet.py` updates the value field. You must also manually update the last point in the corresponding `_frozen_historical` array to keep the chart in sync. The last point in every macro historical array is the 2026F forecast. This convention was established on March 10, 2026.

Sheet URL (published CSV):

https://docs.google.com/spreadsheets/d/e/2PACX-1vQgdfggKVeP6013Pctc3_L_hJGLE--b9jiGaU-yMHwKK_i05o4lPg4dxHvq1hl03uTb-q

Columns synced: GDP_Growth_2026, Inflation_2026, Budget_Deficit_2026, Current_Account_2026, Unemployment_2026, Policy_Rate_2026.

What the sheet does NOT control: Market metrics (handled by `fetch_market_data.py`) or stories (handled by `update_stories.py`). Commodity data, global stories, metricBriefs, and historical chart data are updated manually or via dedicated scripts.

How to view the app locally

Open directly in Safari (no server needed):

```
open -a Safari ~/Downloads/macrosnaps/macrosnaps-globe.html
```

Or run a local server and open in any browser:

```
cd ~/Downloads/macrosnaps && python3 -m http.server 8000
```

Then go to <http://localhost:8000/macrosnaps-globe.html>. Keep that terminal window open while browsing.

Current content state (March 11, 2026)

Per-metric stories (beginner / moderate / expert)

All 168 per-metric stories are complete across all 12 countries and all 3 levels. Stories for the 4 data-void metrics (Equity Vol, Corp Spread, Sov CDS, FX Vol) will not be maintained going forward as those metrics are moving to "not available" display state.

Other content (all 12 countries)

- Country-level stories (3 bullets per level): complete for all 12, last updated March 10, 2026
- metricBriefs (short summaries per metric): complete for all 12
- fxRegime descriptions (3 levels): complete for all 12

Commodity stories (beginner / moderate / expert)

All 9 commodities have a story object with beginner, moderate, and expert keys. Commodity prices are updated automatically by `fetch_market_data.py` daily. Stories are now also updated automatically by

update_commodity_stories.py, which runs daily and rewrites any commodity whose price has moved past threshold (10% for Natural Gas, 5% for all others). All 9 stories bootstrapped and storywrittenAtPrice set on March 12, 2026. No manual story session needed unless you want to override the automated output.

Global stories (March 10, 2026)

- Slot 1: Oil Swings Wildly as Iran War Dominates Markets (WTI \$119 to \$88 intraday)
- Slot 2: US Lost 92,000 Jobs in February (stagflation signal, Fed stuck)
- Slot 3: Stock Markets Bounce Back on Peace Hopes (S&P +0.8%, Gold \$5,145)

Known value/story drift (as of March 11, 2026)

These are metrics where the live value has been updated by fetch_market_data.py but the stories still reference older numbers. Use the Stories Session Prompt in Part 1B to clear this backlog. Do not fix these manually outside a dedicated story session.

USA - values as of March 11 dry run:

Metric	Current value	What stories say
Stock Market YTD	-1.1%	+2%
Equity Vol (VIX)	~25	~16
10Y Bond Yield	4.12%	4.28%
Yield Curve	+52bps	+8bps
Corp Spread	84bps	85bps
USD/DXY	99.1	104.2
FX Vol	5.8%	8.5%
Policy Rate	Rewritten March 11, 2026	
Budget Deficit	Rewritten March 11, 2026	

CAN - values as of March 11 dry run:

Metric	Current value	What stories say
GDP Growth	Rewritten March 11, 2026	
Stock Market YTD	+4.1%	6.8%
10Y Bond Yield	3.40%	(check story)
Yield Curve	+121bps	+30bps
CAD/USD	0.74	(check story)
FX Vol	5.2%	7.2%

Other countries - live values as of March 11 dry run:

Country	Metric	Current value
GBR	Stock Market YTD	+3.8%
GBR	10Y Bond Yield	4.45%
GBR	Yield Curve	+74bps
GBR	GBP/USD	1.34
JPN	Stock Market YTD	Rewritten March 11, 2026
JPN	Equity Vol	~32
JPN	10Y Bond Yield	2.24%
JPN	Yield Curve	+112bps
JPN	USD/JPY	158.6

Country	Metric	Current value
DEU	Stock Market YTD	-3.8%
DEU	10Y Bond Yield	2.81%
DEU	Yield Curve	+81bps
DEU	EUR/USD	1.1589
FRA	Stock Market YTD	-2.2%
FRA	10Y Bond Yield	3.53%
FRA	Yield Curve	+153bps
ITA	Stock Market YTD	-1.4%
ITA	10Y Bond Yield	3.49%
ITA	Yield Curve	+149bps
CHN	Stock Market YTD	+2.7%
CHN	USD/CNY	6.86
IND	Stock Market YTD	-9.8%
IND	10Y Bond Yield	6.73%
IND	Yield Curve	+123bps
IND	Sov CDS	261bps
IND	USD/INR	92.04
ZAF	Stock Market YTD	+0.5%
ZAF	Equity Vol	~29
ZAF	10Y Bond Yield	8.62%
ZAF	Yield Curve	+187bps
ZAF	Sov CDS	450bps
ZAF	USD/ZAR	16.38
BRA	Stock Market YTD	+15.1%
BRA	USD/BRL	5.16
RUS	USD/RUB	79.1

Minor rounding gaps only (not material): CHN GDP, IND GDP, ZAF GDP. All within 0.2-0.5pp of story figure. Low priority.

fetch_market_data.py

The script lives in ~/Downloads/macrosnaps/. It pulls all 8 market metrics and writes value and last_updated fields directly into data.json. It never touches historical arrays, stories, macro metrics, or any other field.

Metrics fetched:

Metric	Source	Notes
Stock Market YTD	Yahoo Finance	YTD % change from Jan 1 close
Equity Vol	Yahoo Finance	Implied vol index where available, 30-day realized vol as fallback
10Y Bond Yield	FRED	Daily series per country
Yield Curve	FRED (derived)	10Y minus short rate
Corp Spread	FRED	ICE BofA IG/HY OAS series
Sov CDS	Derived proxy	Local 10Y minus UST (EM countries only)
FX pair	Yahoo Finance	Varies by country
FX Vol	Yahoo Finance (derived)	30-day realized vol from daily FX returns

Metric	Source	Notes
Commodity prices (9)	Yahoo Finance	Continuous futures tickers (CL=F, BZ=F, NG=F, GC=F, SI=F, HG=F, ZW=F, ZC=F, ZS=F); updates price, change (YoY %), spark (rolling 12-point array), and asOf

Known gaps (expected, not bugs):

- RUS: IM0EX.ME is delisted on Yahoo. Stock Market YTD and Equity Vol will always fail. FX and FX Vol come through.
- CHN and BRA: No FRED 10Y series. 10Y Bond Yield, Yield Curve, and Sov CDS will always fail.
- RUS: No Corp Spread series configured.
- Bund 10Y pre-fetch returns unrounded (e.g. 2.80666666666667%). Minor cosmetic issue, does not affect output values.

Dry run results (March 11, 2026): 77 of 96 metrics fetched successfully. 19 failures all fall in the known gaps above.

Requirements (one-time setup):

```
pip3 install requests yfinance python-dotenv
```

FRED API key (free): https://fred.stlouisfed.org/docs/api/api_key.html

The .env file lives at ~/Downloads/macrosnaps/.env and contains:

```
FRED_API_KEY=your_key_here
```

This file exists as of March 11, 2026.

To run (dry run first):

```
cd ~/Downloads/macrosnaps
python3 fetch_market_data.py --dry-run
python3 fetch_market_data.py
```

After a successful live run:

```
python3 build.py && git add -A && git commit -m "Daily market update $(date +%Y-%m-%d)" && git push origin master
```

Historical chart data state (March 10, 2026)

Macro metric charts - structure as of March 10:

All macro historical arrays follow the same pattern: 6 points covering 2020-2025 actuals, plus a final 7th point which is the 2026F forecast from the Google Sheet. Current Account arrays were converted from absolute USD values to % of GDP on March 10 using IMF WEO denominators. When forecasts change in the sheet, update both the value field and the last point of the corresponding _frozen_historical array.

GDP denominators used for Current Account conversion (IMF WEO, USD billions, 2020-2025):

Country	2020	2021	2022	2023	2024	2025
USA	21,060	23,315	25,744	27,357	29,168	30,000
CAN	1,647	1,988	2,140	2,140	2,178	2,150
GBR	2,698	3,131	3,071	3,082	3,341	3,450
JPN	5,040	4,940	4,232	4,213	4,109	4,390
DEU	3,888	4,257	4,073	4,455	4,526	4,600
FRA	2,703	2,957	2,786	3,031	3,130	3,200
ITA	1,889	2,107	2,010	2,254	2,300	2,380
CHN	14,688	17,734	17,963	17,795	18,530	19,500
IND	2,671	3,150	3,386	3,730	3,943	4,270
ZAF	335	420	405	378	373	390
BRA	1,449	1,649	1,921	2,174	2,330	2,100

Country	2020	2021	2022	2023	2024	2025
RUS	1,483	1,829	2,241	1,914	2,100	2,100

Per-country chart population (_frozen_historical):

Country	Charts populated	Notes
USA	14/14	Yield Curve fixed 2026-03-09; CA converted to % GDP 2026-03-10
CAN	14/14	CA converted to % GDP 2026-03-10; Budget Deficit array empty
GBR	14/14	CA converted to % GDP 2026-03-10; Budget Deficit array empty
JPN	14/14	Yield Curve fixed 2026-03-09; CA converted to % GDP 2026-03-10
DEU	14/14	Yield Curve fixed 2026-03-09; CA converted to % GDP 2026-03-10; Budget Deficit array empty
FRA	14/14	Yield Curve fixed 2026-03-09; CA converted to % GDP 2026-03-10; Budget Deficit array empty
ITA	14/14	CA converted to % GDP 2026-03-10
CHN	11/14	Unemployment, 10Y Bond Yield, Yield Curve: no free source; Budget Deficit array empty; CA converted to % GDP 2026-03-10
IND	11/14	Unemployment, 10Y Bond Yield, Yield Curve: no free source; Budget Deficit array empty; CA converted to % GDP 2026-03-10
ZAF	12/14	Unemployment: no monthly source; Yield Curve populated; Budget Deficit array empty; CA converted to % GDP 2026-03-10
BRA	12/14	10Y Bond Yield, Yield Curve: no free source; Budget Deficit array empty; CA converted to % GDP 2026-03-10
RUS	10/14	GDP Growth, Unemployment, USD/RUB discontinued post-2022 sanctions; Budget Deficit array empty; CA converted to % GDP 2026-03-10

Known data quality flags:

- IND and ZAF GDP Growth historical arrays show anomalously large values in some years (IND 2025: 16.77, ZAF 2025: 6.76), likely reflecting nominal USD growth from FRED rather than real % growth. The 2026F forecast has been appended as the correct final point. The historical points are a pre-existing issue to investigate separately.
- Budget Deficit arrays are empty for 9 countries (CAN, GBR, DEU, FRA, CHN, IND, ZAF, BRA, RUS). A single forecast point with no history would be meaningless as a chart, so these were left empty intentionally.

The 14 metrics per country

Macro (6): GDP Growth, Inflation (CPI), Unemployment, Budget Deficit, Current Account, Policy Rate.

Market (8): Stock Market YTD, Equity Vol, 10Y Bond Yield, Yield Curve, Corp Spread, Sov CDS, [FX pair - varies by country], FX Vol.

Data-void metrics (4): Equity Vol, Corp Spread, Sov CDS, FX Vol. No reliable free daily source exists for any of these. These will be displayed as "not available" in the UI with a tooltip explanation. Stories for these metrics will not be maintained going forward. This decision can be revisited post-launch if a paid data source is added.

How metric stories work

Stories live in `data.json` inside each metric entry as a story object with `beginner`, `moderate`, and `expert` keys.

At load time the shell reads these into the `metricStories` object (line 5605 of shell).

When a user clicks a metric, `renderMTT()` (line 6542) builds the tooltip. The story appears inline between the value and the chart, with no header label. CSS class is `tt-metric-story` (line 165 of shell).

The tooltip order is: metric name, country + value, story, chart, explanation/bluf, FX regime (if applicable), compare button.

Stories are written and maintained by AI. `update_stories.py` (built March 11, 2026) diffs `data.json` against the last git commit, identifies metrics that changed past a configurable threshold, calls the Claude API, and rewrites the affected stories at all three levels in one pass. Do not write or edit stories by hand outside a dedicated stories session.

Architecture decisions and why

Google Sheet for macro metrics. The sheet holds the 6 macro metrics per country. `sync_sheet.py` pulls from the sheet and writes `data.json`. The sheet is updated manually but infrequently, when year-end consensus forecasts change.

`fetch_market_data.py` for live market metrics. Pulls all 8 market metrics from Yahoo Finance and FRED daily. Built and verified March 11, 2026.

`update_stories.py` for story maintenance. Diffs `data.json` against the last git commit to detect meaningful value changes, then calls the Claude API to rewrite stories for affected metrics. Runs after both `sync_sheet.py` and `fetch_market_data.py` so it catches all changes in one pass. Built and verified March 11, 2026. Requires `ANTHROPIC_API_KEY` in `.env`.

`update_commodity_stories.py` for commodity story maintenance. Runs daily after `fetch_market_data.py`. Compares each commodity's current price field against `storyWrittenAtPrice` stored in `data.json`. Rewrites stories at all three levels for any commodity that has moved past its threshold (10% for Natural Gas, 5% for all others). Applies directly to `data.json` with no draft or review step. Built and verified March 12, 2026.

No CMS. Pre-launch solo workflow. JSON plus build script is faster to iterate with than any external system.

GitHub Pages for hosting. Repo is public at <https://github.com/ralphlazar/macrosnaps>. Deploy from master branch, root folder. The built HTML file is self-contained (data inlined) so no build step is needed on the server side.

Git for version history. Commit after every feature or meaningful change. `update_stories.py` uses `git diff` to detect what has changed since the last commit, so committing consistently is essential for correct story rewrite targeting.

Vulnerabilities and mitigations

Vulnerability	Status	Mitigation
No version history	Fixed	Git initialized. Commit after every build.
JSON corruption from manual edits	Fixed	Run <code>python3 build.py --validate-only</code> before editing. Build script also validates on every run.
Forgetting to rebuild	Habit-dependent	Never call a task done until build has run and output copied.
No preview step	Fixed	<code>sync_sheet.py</code> preview mode shows all changes before anything is written.
Headline stories applied without review	Fixed	<code>update_headlines.py</code> writes a draft file only. <code>headline_review.html</code> is a browser tool for reviewing, editing, and approving before applying.
Spurious git diffs from build timestamps	Fixed	Build script stamps <code>_meta</code> in memory only. <code>data.json</code> on disk is never written by the build.

Vulnerability	Status	Mitigation
Daily backup only	Covered	Git provides full history. Build script saves daily backup to backups/ and prunes after 30 days.
Historical chart data incomplete	Largely resolved	refetch_historical.py restored all charts except genuine data voids.
Stale or missing market data	Resolved	fetch_market_data.py built and verified March 11, 2026. 4 data-void metrics display as "not available." Commodity prices automated March 12, 2026.
Manual story maintenance burden	Resolved	update_stories.py built March 11, 2026. Rewrites stories for any metric that moves past threshold.
Weather Map showing stale forecast values	Fixed	Shell reads the 2026F column from live metrics at runtime. Historical years stay frozen.
Value/story drift between daily market updates and story rewrites	Resolved	update_stories.py built March 11, 2026. Run after each data fetch to keep stories current.
Current Account displayed in absolute dollars	Fixed 2026-03-10	All CA historical arrays converted to % of GDP using IMF WEO denominators. 2026F appended as final point.
Macro chart 2026F point missing	Fixed 2026-03-10	2026F forecast appended as final point to all macro historical arrays with existing data.
refetch_historical.py overwrites CA % GDP conversion	Known risk	Running refetch restores raw dollar values. Re-run CA conversion after any refetch. Fix properly when updating refetch_historical.py (see Pending work).
No .env file	Fixed 2026-03-11	.env created at ~/Downloads/macrosnaps/.env with FRED_API_KEY.
.env committed to public repo	Resolved 2026-03-11	.env and env removed from all git history via filter-branch, force pushed. .env and env added to .gitignore. Anthropic API key regenerated twice (exposed in git history, then again in chat). Always use interactive input or nano to set secrets, never paste into chat.

Key shell locations

Thing	Location
.tt-metric-story CSS	Line 165
metricStories object declaration	Line 5605
parseLiveVal() helper	Line 5613
metricStories populated from data	Line 5637
renderMTT() function	Line 6542
Metric story HTML built	Line 6581-6582
Full tooltip render string	Line 6589
historicalData populated from _frozen_historical	Line 5638
Weather grid population (live override)	Line 5646
window.__MACROSNAPS_DATA__ injected by build	Before </head> in built output

update_commodity_stories.py

The script lives in ~/Downloads/macrosnaps/. It runs as part of the daily ritual after fetch_market_data.py. It checks each commodity's current price against storyWrittenAtPrice in data.json and rewrites stories for any that have moved past threshold.

Thresholds:

Commodity	Threshold
Natural Gas	10%
All others	5%

Fields written to each commodity in data.json:

- story.beginner.text, story.moderate.text, story.expert.text - rewritten story
- storyWrittenAtPrice - price at time of last write, used for future drift checks
- storyUpdatedDate - datestamp of last rewrite

First-run behaviour: Any commodity missing storyWrittenAtPrice is treated as needing a rewrite, bootstrapping the state. All 9 commodities bootstrapped March 12, 2026.

Requirements:

```
pip3 install anthropic python-dotenv
```

ANTHROPIC_API_KEY must be set in .env.

To run:

```
cd ~/Downloads/macrosnaps
python3 update_commodity_stories.py
```

refetch_historical.py

The script lives in ~/Downloads/macrosnaps/. It pulls data from FRED and Yahoo Finance and writes _frozen_historical into data.json in place. It never touches any other field.

Warning: Running refetch_historical.py will overwrite Current Account arrays with raw absolute dollar values, undoing the % GDP conversion. After any refetch run, the CA conversion must be re-applied. This is a known risk until refetch_historical.py is updated to output % of GDP natively (see Pending work).

Requirements (one-time setup):

```
pip3 install requests yfinance python-dotenv
```

To run:

```
cd ~/Downloads/macrosnaps && python3 refetch_historical.py
```

After a successful run:

```
python3 build.py && git add -A && git commit -m "Restore _frozen_historical"
git push origin master
```

Key settings at the top of the script:

- FORCE_OVERWRITE = False - set to True to re-fetch countries that already have data
- MIN_POINTS = 5 - existing point count that qualifies as already populated

Pending work (priority order)

1. **Commodity story rewrite session.** Done March 12, 2026. update_commodity_stories.py built and added to daily ritual. Bootstrapped all 9 stories on first run. Prices have moved significantly since March 10 (Gold +74% YoY, Silver +157% YoY, WTI at \$93.61, Natural Gas down 22% YoY) - all now rewritten automatically.

2. **Apply USA sheet changes.** `sync_sheet.py` preview on March 11 showed USA CPI 3.1% -> 2.3% and Unemployment 4.4% -> 4.2%. These were never applied. Run: `python3 sync_sheet.py --apply && python3 update_stories.py && python3 build.py`

2. **Manual stories session.** Use the Stories Session Prompt in Part 1B to rewrite all stale metric stories listed in "Known value/story drift" above. This is a content session - upload `LIVING_BRIEF.md` + `data.json`. Do this before building `update_stories.py` so there is a clean baseline.

2. **Grey out 4 data-void metrics in the UI.** Equity Vol, Corp Spread, Sov CDS, and FX Vol should display as "not available" with a tooltip: "no reliable free data source - will be added post-launch." This is a UI session - upload `LIVING_BRIEF.md` + `macrosnaps-shell.html`.

3. **Build `update_stories.py`.** Done March 11, 2026. Script diffs `data.json` against last git commit, applies per-metric thresholds, calls Claude API (claude-sonnet-4-20250514), and writes all three story levels back into `data.json`. Also rewrites stories for data-void metrics if their values are ever populated.

4. **Update `refetch_historical.py` to output Current Account as % of GDP natively.** Running `refetch` currently overwrites CA arrays with raw dollar values, requiring a manual re-conversion. The script should divide by nominal GDP at fetch time. This is a tooling session.

5. **Investigate IND and ZAF GDP Growth historical anomalies.** Some historical points appear to reflect nominal USD growth rather than real % growth (IND 2025: 16.77, ZAF 2025: 6.76). Verify the FRED series and correct if needed. This is a tooling session.

6. **Build `print_snapshot.py`.** Uses Playwright to open the built HTML file, loops through each country, expands it, and captures a full-height PDF. Output is a dated file in `snapshots/` (e.g. `macrosnaps-2026-03-09.pdf`). Audience level hardcoded to expert. For personal use only, not a public feature. Requires `pip3 install playwright` and `playwright install chromium`.

Global stories narrative formula

The three global story cards always follow a fixed three-act arc:

- Card 1 - Today's Story: the dominant macro event of the day
- Card 2 - Biggest Movers: which markets, currencies, or economies are reacting and how
- Card 3 - The Connection: what ties cards 1 and 2 together, the "so what" for the global picture

All three levels (beginner, moderate, expert) tell the same arc at different depths. The icon and label are identical across levels. This formula is enforced in the `build_global_system()` prompt in `update_headlines.py`.

Editorial principle: forecasts vs stories

MacroSnaps has two distinct layers of truth that must not be conflated:

Forecast values (source: Ralph's Google Sheet) are annual consensus views for 2026. They drive the metric value displayed on each card and the weather icon. Policy Rate is the year-end forecast. These are proprietary and intentionally stable.

Stories should be written off recent data and trends, not off the forecast values. Monthly CPI prints, quarterly GDP flash estimates, central bank decisions, weekly jobless claims - this is the live texture that makes stories worth reading. A story that just restates the annual forecast number adds no value.

The correct approach: stories comment on what is actually happening right now. If recent data is tracking ahead of or behind the annual forecast, the story can note that tension briefly (e.g. "February CPI came in at 2.6%, above the Fed target, but full-year inflation is still expected to settle at 2.3% as base effects kick in mid-year"). But the forecast is not the anchor of the story - recent data is.

Implication for tooling: implemented March 12, 2026. See `update_headlines.py` architecture: Sonnet+search harvest call feeds recent data into Haiku writing batches as lead context.

7. Add country-level and global stories to the daily bash ritual. Done and verified March 11, 2026.

`update_headlines.py` calls the Claude API, drafts 12 country story blocks (3 bullets x 3 levels) in 3 batches of 4 via Haiku (8000 tokens each) and global stories (3 items x 3 levels) via Sonnet with web search (5000 tokens). Saves to `stories_draft_YYYY-MM-DD.json`. `headline_review.html` is a browser-based review tool: tabs show Bullet 1/2/3, each tab shows all three levels stacked for easy comparison. Export `stories_approved_YYYY-MM-DD.json`, apply with `python3 update_headlines.py --apply`. First live run completed 13/13 March 11, 2026.

8. Enable web search for country stories in `update_headlines.py`. Done March 12, 2026. Two-phase architecture: a single Sonnet+search harvest call pulls recent data for all 12 countries (capped at 2 search turns, 1500 max_tokens), then 3 Haiku batches write stories with recent data leading and forecast values passed as background context only. Global runs first to avoid Sonnet rate limit exhaustion from the harvest call. Haiku prompt strengthened with explicit bullet count enforcement and one automatic retry per batch.

9. Post-launch: revisit architecture if a second person joins to update data daily.